

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет: Компьютерных систем и сетей

Кафедра: Электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЁТ
к лабораторной работе № 1
на тему
«ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ ПО.
КАЛЬКУЛЯТОР ЖИЗНИ»
Вариант 9

Студент:

А.С. Гурьянов

Преподаватель:

О.М. Внук

Минск 2026

1 ЦЕЛЬ И ЗАДАЧИ

Цель работы – изучить методы работы с интегрированной средой разработки Visual Studio Code при создании объектно-ориентированного приложения на языке C++ с использованием библиотеки Qt 6, освоить запуск и отладку программы с графическим интерфейсом, а также применить архитектурный паттерн MVC с активной моделью.

В соответствии с вариантом 9 необходимо разработать программу «Калькулятор Жизни», которая по введённой дате рождения вычисляет статистические показатели прожитой жизни (время сна, количество морганий, ударов сердца, объём перекаченной крови, объём выпитой воды, количество смеха). Для выполнения работы поставлены следующие задачи:

1. Создать и настроить проект C++ в Visual Studio Code с использованием системы сборки CMake, изучить средства редактирования, запуска и отладки;
2. Установить расширения для работы с UML и SysML v2;
3. Разработать графическое приложение с главной формой и отдельным окном ввода данных;
4. Реализовать в программе сохранение последних корректных данных и их восстановление при повторном вводе;
5. Реализовать проверку входных данных и вывод понятного сообщения при вводе некорректных данных;
6. Разделить приложение на модель, представление и контроллер, а также реализовать активное уведомление представления моделью;

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЕ

2.1 Интегрированная среда разработки и её компоненты

Visual Studio Code — это легковесная, но расширяемая среда разработки. В отличие от классических IDE (например, Visual Studio, Qt Creator) VS Code по умолчанию является редактором кода, который преобразуется в полноценную IDE за счет многочисленных расширений. Для разработки на C++ с Qt использовались расширения C/C++ и CMake Tools, которые обеспечивают подсветку синтаксиса, анализ кода, интеграцию с отладчиком, поддерживающим пошаговую отладку с точками останова, настройку, конфигурацию, сборку и запуск проекта.

2.2 Редактирование и навигация в Visual Studio Code

Редактор VS Code поддерживает автодополнение на основе IntelliSense, быстрый переход к объявлению символа, поиск всех использований, переименование символа, форматирование документа. Структура проекта отображается в боковой панели Explorer, сообщения компилятора и сборки — в панели Output, проблемы — в панели Problems. Палитра команд предоставляет доступ ко всем функциям редактора и установленных расширений.

2.3 Сборка и запуск приложения

C++ компилируется непосредственно в машинный код. В работе используется система сборки CMake, которая генерирует файлы сборки (Makefile/Ninja) на основе файла CMakeLists.txt. Сборка выполняется командой `cmake --build`, запуск — через CMake Tools или напрямую из терминала.

2.4 Паттерн MVC

MVC (Model-View-Controller) — это архитектурный паттерн, разделяющий приложение на три компонента:

- Model — хранит состояние приложения и выполняет предметную (бизнес-)логику;
- View — формирует графический интерфейс и отображает данные пользователю;
- Controller — обрабатывает действия пользователя (клики, ввод данных) и передаёт команды модели, такое разделение уменьшает связанность

классов, упрощает тестирование и позволяет изменять интерфейс без затрагивания логики.

В варианте с пассивной моделью контроллер после изменения данных сам инициирует обновление представления, выступая в роли «курьера» между моделью и view.

Активная модель самостоятельно сообщает подписчикам об изменении своего состояния, используя паттерн Observer. В разработанном приложении LifeModel реализует активную модель через внутренний интерфейс ModelObserver и метод notifyObservers(), который автоматически вызывается после выполнения расчётов.

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Постановка задачи

Разработана утилита «Калькулятор Жизни», которая принимает дату рождения (день, месяц, год — через три отдельных поля) и вычисляет шесть статистических показателей прожитой жизни:

- количество часов и суток сна (из расчёта 8 часов в сутки);
- количество морганий (15 раз в минуту × 16 часов бодрствования);
- количество ударов сердца (75 ударов в минуту);
- объём перекачанной сердцем крови (70 мл за один удар);
- объём выпитой воды (2,5 литра в сутки);
- количество раз смеха (17 раз в сутки).

На главной форме размещена кнопка «Ввести данные», открывающая окно `InputDialog`. Реализована поддержка восстановления последних введённых данных: при открытии окна ввода подставляются значения из предыдущего успешного расчёта.

3.2 Структура проекта и каркас MVC с активной моделью

Проект разделён на следующие компоненты:

- `LifeModel` — хранит дату рождения, результаты расчёта и последние введённые данные, содержит бизнес-логику вычислений и механизм уведомления слушателей (активная модель);
- `MainWindow` — главный интерфейс, реализует интерфейс `ModelObserver` для автоматического обновления при изменении данных;
- `InputDialog` — окно для сбора даты;
- `LifeController` — обрабатывает ввод пользователя, открывает диалог ввода, передаёт данные в модель и отображает ошибки;

`LifeModel` реализует активную модель. Метод `calculate` выполняет проверку валидности даты. После успешной валидации выполняются расчёты всех шести величин, и в конце метода вызывается `notifyObservers()` — ключевой метод активной модели, который оповещает всех подписчиков об изменении данных.

`MainWindow` отображает шесть рассчитанных величин. Благодаря реализации интерфейса `ModelObserver`, метод `onModelChanged()` обновляет текстовые метки мгновенно после того, как модель зафиксировала изменения — без явного вызова со стороны контроллера.

`InputDialog` обеспечивает ввод даты через три независимых.

`LifeController` управляет жизненным циклом `InputDialog` и отвечает за передачу пользовательского ввода в модель. Контроллер связывается с представлением через механизм сигналов и слотов Qt: сигнал `MainWindow::openInputRequested()` соединяется со слотом `LifeController::onOpenInputDialog()`.

4 РЕЗУЛЬТАТЫ РАБОТЫ

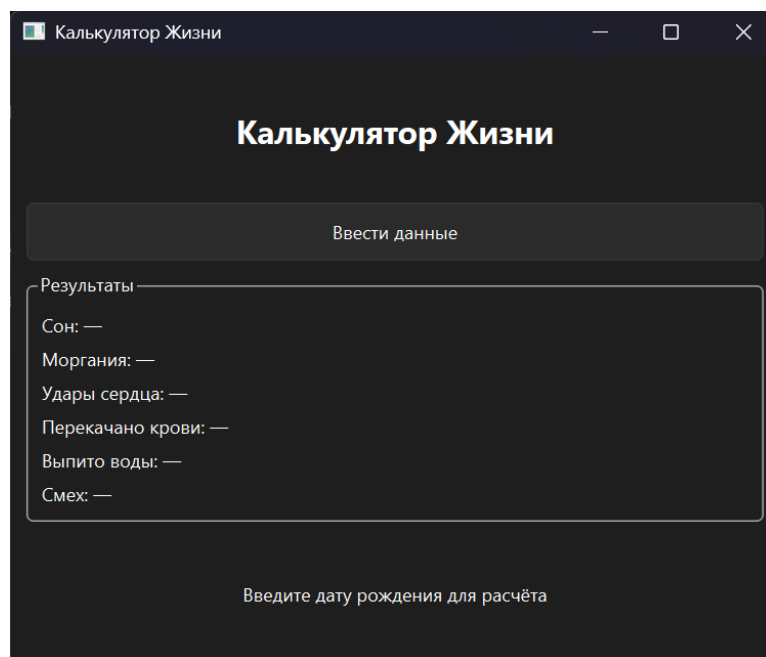


Рисунок 4.1 – Главная форма после запуска

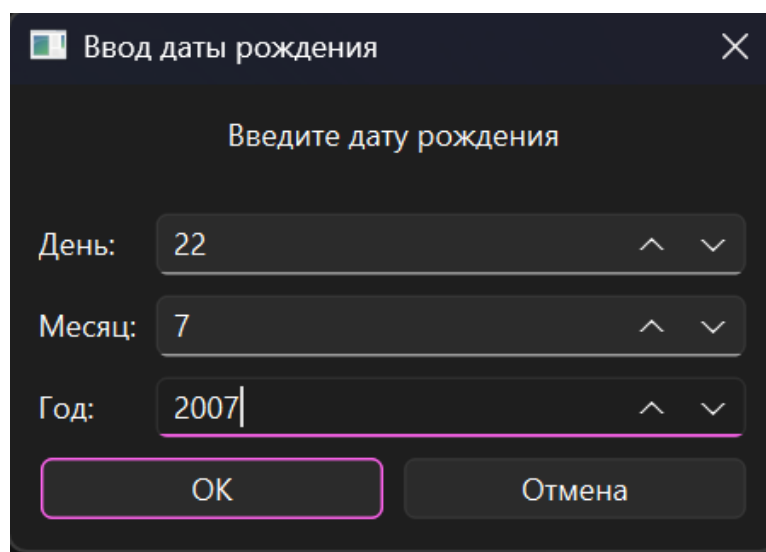


Рисунок 4.2 – Окно ввода данных

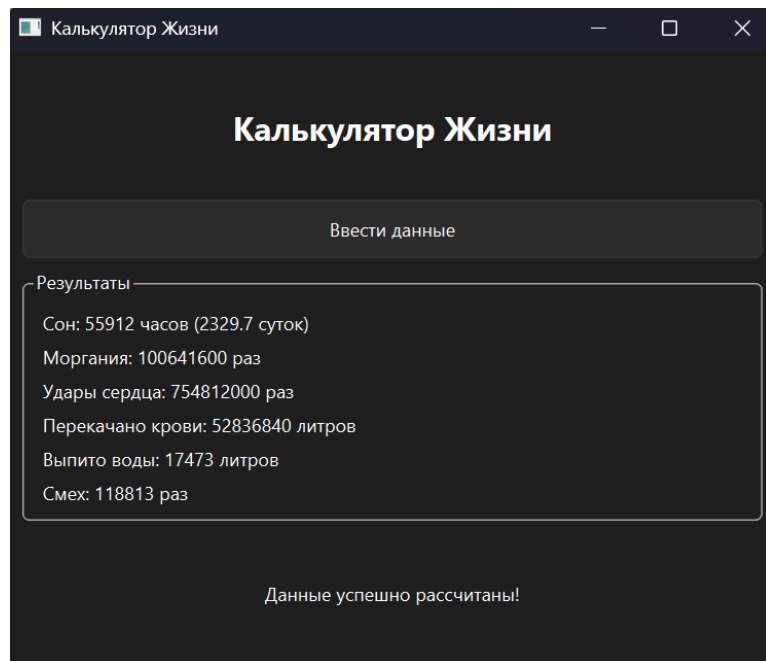


Рисунок 4.3 – Результат вычисления

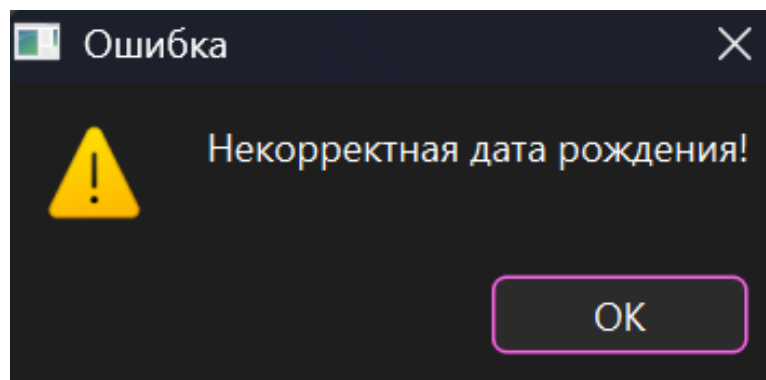


Рисунок 4.4 – Сообщение об ошибке

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены основные возможности Visual Studio Code для редактирования, сборки и запуска C++ - приложения с использованием библиотеки Qt 6 и системы сборки CMake.

Разработана утилита «Калькулятор Жизни» с графическим интерфейсом на. Программа вычисляет шесть статистических показателей (сон, моргания, сердце, кровь, вода, смех), поддерживает запоминание и восстановление последних введенных данных, сообщает о некорректном вводе через. Компоненты разделены по паттерну MVC, а активная модель уведомляет представление о новом состоянии с помощью паттерна Наблюдатель (ModelObserver).

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг исходного кода приложения

inputdialog.cpp

```
#include "inputdialog.h"
#include <QSpinBox>
#include <QPushButton>
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QLabel>
#include <QFormLayout>

InputDialog::InputDialog(QWidget *parent) : QDialog(parent) {
    setWindowTitle("Ввод даты рождения");
    setMinimumWidth(300);

    QVBoxLayout *mainLayout = new QVBoxLayout(this);

    QLabel *info = new QLabel("Введите дату рождения\n", this);
    info->setAlignment(Qt::AlignCenter);
    mainLayout->addWidget(info);

    QFormLayout *formLayout = new QFormLayout;

    m_daySpin = new QSpinBox(this);
    m_daySpin->setRange(1, 31);
    m_daySpin->setValue(1);

    m_monthSpin = new QSpinBox(this);
    m_monthSpin->setRange(1, 12);
    m_monthSpin->setValue(1);

    m_yearSpin = new QSpinBox(this);
    m_yearSpin->setRange(1900, 2026);
    m_yearSpin->setValue(2000);

    formLayout->addRow("День:", m_daySpin);
    formLayout->addRow("Месяц:", m_monthSpin);
    formLayout->addRow("Год:", m_yearSpin);

    mainLayout->addLayout(formLayout);

    QHBoxLayout *buttonLayout = new QHBoxLayout;
    m_okButton = new QPushButton("OK", this);
    m_cancelButton = new QPushButton("Отмена", this);

    buttonLayout->addWidget(m_okButton);
    buttonLayout->addWidget(m_cancelButton);
```

```

        mainLayout->addLayout(buttonLayout);

        connect(m_okButton, &QPushButton::clicked, this, &QDialog::accept);
        connect(m_cancelButton, &QPushButton::clicked, this,
&QDialog::reject);
    }

void InputDialog::setInitialValues(int day, int month, int year) {
    m_daySpin->setValue(day);
    m_monthSpin->setValue(month);
    m_yearSpin->setValue(year);
}

int InputDialog::day() const { return m_daySpin->value(); }
int InputDialog::month() const { return m_monthSpin->value(); }
int InputDialog::year() const { return m_yearSpin->value(); }

```

inputdialog.h

```

#ifndef INPUTDIALOG_H
#define INPUTDIALOG_H

#include <QDialog>

class QSpinBox;
class QPushButton;

class InputDialog : public QDialog {
    Q_OBJECT

public:
    explicit InputDialog(QWidget *parent = nullptr);

    void setInitialValues(int day, int month, int year);

    int day() const;
    int month() const;
    int year() const;

private:
    QSpinBox *m_daySpin;
    QSpinBox *m_monthSpin;
    QSpinBox *m_yearSpin;
    QPushButton *m_okButton;
    QPushButton *m_cancelButton;
};

#endif

```

lifecontroller.cpp

```

#include "lifecontroller.h"
#include "lifemodel.h"
#include "mainwindow.h"
#include "inputdialog.h"
#include <QDate>
#include <QMessageBox>

LifeController::LifeController(LifeModel *model, MainWindow *view)
    : m_model(model), m_view(view) {
    connect(m_view, &MainWindow::openInputRequested,
           this, &LifeController::onOpenInputDialog);
}

void LifeController::onOpenInputDialog() {
    InputDialog dialog(m_view);

    if (m_model->hasLastInput()) {
        dialog.setInitialValues(
            m_model->lastDay(),
            m_model->lastMonth(),
            m_model->lastYear()
        );
    }

    if (dialog.exec() == QDialog::Accepted) {
        int day = dialog.day();
        int month = dialog.month();
        int year = dialog.year();

        onInputConfirmed(day, month, year);
    }
}

void LifeController::onInputConfirmed(int day, int month, int year) {
    m_model->setLastInput(day, month, year);

    QDate birthDate(year, month, day);
    QString error;

    if (!m_model->calculate(birthDate, error)) {
        QMessageBox::warning(m_view, "Ошибка", error);
        return;
    }

    m_view->showSuccessMessage("Данные успешно рассчитаны!");
}

```

lifecontroller.h

```

#ifndef LIFECONTROLLER_H

```

```

#define LIFECONTROLLER_H

#include <QObject>

class LifeModel;
class MainWindow;

class LifeController : public QObject {
    Q_OBJECT

public:
    LifeController(LifeModel *model, MainWindow *view);

public slots:
    void onOpenInputDialog();
    void onInputConfirmed(int day, int month, int year);

private:
    LifeModel *m_model;
    MainWindow *m_view;
};

#endif

```

lifemodel.cpp

```

#include "lifemodel.h"
#include <QDate>
#include <QDateTime>

LifeModel::LifeModel(QObject *parent) : QObject(parent) {}

void LifeModel::addObserver(ModelObserver *observer) {
    if (observer && !m_observers.contains(observer)) {
        m_observers.append(observer);
    }
}

void LifeModel::removeObserver(ModelObserver *observer) {
    m_observers.removeOne(observer);
}

void LifeModel::setLastInput(int day, int month, int year) {
    m_lastDay = day;
    m_lastMonth = month;
    m_lastYear = year;
    m_hasLastInput = true;
}

int LifeModel::lastDay() const { return m_lastDay; }

```

```

int LifeModel::lastMonth() const { return m_lastMonth; }
int LifeModel::lastYear() const { return m_lastYear; }
bool LifeModel::hasLastInput() const { return m_hasLastInput; }

bool LifeModel::calculate(const QDate &birthDate, QString &error) {
    QDate today = QDate::currentDate();

    if (!birthDate.isValid()) {
        error = "Некорректная дата рождения!";
        return false;
    }
    if (birthDate > today) {
        error = "Дата рождения не может быть в будущем!";
        return false;
    }

    qint64 daysLived = birthDate.daysTo(today);
    if (daysLived <= 0) {
        error = "Некорректная дата!";
        return false;
    }

    // СТАТИСТИКА (средние значения)
    // Сон: 8 часов в сутки
    m_sleepHours = daysLived * 8.0;
    m_sleepDays = m_sleepHours / 24.0;

    // Моргания: 15 раз в минуту в бодрствование (16 часов)
    // 15 * 60 * 16 = 14,400 раз в день
    m_blinks = daysLived * 14400LL;

    // Удары сердца: 75 ударов в минуту
    // 75 * 60 * 24 = 108,000 ударов в день
    m_heartBeats = daysLived * 108000LL;

    // Кровь: 70 мл за один удар (ударный объём)
    // 108,000 * 0.07 = 7,560 литров в день
    m_bloodLiters = daysLived * 7560.0;

    // Вода: 2.5 литра в день
    m_waterLiters = daysLived * 2.5;

    // Смех: 17 раз в день (взрослые)
    mLaughs = daysLived * 17LL;

    notifyObservers();

    return true;
}

void LifeModel::notifyObservers() {

```

```

        for (ModelObserver *observer : m_observers) {
            observer->onModelChanged();
        }
        emit dataChanged();
    }

double LifeModel::sleepHours() const { return m_sleepHours; }
double LifeModel::sleepDays() const { return m_sleepDays; }
long long LifeModel::blinks() const { return m_blinks; }
long long LifeModel::heartBeats() const { return m_heartBeats; }
double LifeModel::bloodLiters() const { return m_bloodLiters; }
double LifeModel::waterLiters() const { return m_waterLiters; }
long long LifeModel::laughs() const { return mLaughs; }

```

lifemodel.h

```

#ifndef LIFEMODEL_H
#define LIFEMODEL_H

#include <QObject>
#include <QDate>
#include <QList>

class ModelObserver {
public:
    virtual ~ModelObserver() = default;
    virtual void onModelChanged() = 0;
};

class LifeModel : public QObject {
    Q_OBJECT

public:
    explicit LifeModel(QObject *parent = nullptr);

    void addObserver(ModelObserver *observer);
    void removeObserver(ModelObserver *observer);

    void setLastInput(int day, int month, int year);
    int lastDay() const;
    int lastMonth() const;
    int lastYear() const;
    bool hasLastInput() const;

    bool calculate(const QDate &birthDate, QString &error);

    double sleepHours() const;
    double sleepDays() const;
    long long blinks() const;
    long long heartBeats() const;

```

```

        double bloodLiters() const;
        double waterLiters() const;
        long long laughs() const;

signals:
    void dataChanged();

private:
    void notifyObservers();

    QList<ModelObserver*> m_observers;

    int m_lastDay = 0;
    int m_lastMonth = 0;
    int m_lastYear = 0;
    bool m_hasLastInput = false;

    double m_sleepHours = 0;
    double m_sleepDays = 0;
    long long m_blinks = 0;
    long long m_heartBeats = 0;
    double m_bloodLiters = 0;
    double m_waterLiters = 0;
    long long m_laughs = 0;
};

#endif

```

main.cpp

```

#include <QApplication>
#include "mainwindow.h"
#include "lifemodel.h"
#include "lifecontroller.h"

int main(int argc, char *argv[]) {
    QApplication app(argc, argv);
    app.setApplicationName("Калькулятор Жизни");

    LifeModel model;
    MainWindow view;
    view.setModel(&model);

    LifeController controller(&model, &view);

    view.show();
    return app.exec();
}

```

mainwindow.cpp

```
#include "mainwindow.h"
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QLabel>
#include <QPushButton>
#include <QGroupBox>
#include <QMessageBox>

MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent) {
    setWindowTitle("Калькулятор Жизни");
    resize(500, 400);

    m_model = nullptr;
    createUI();
}

void MainWindow::createUI() {
    QWidget *central = new QWidget(this);
    QVBoxLayout *mainLayout = new QVBoxLayout(central);

    QLabel *title = new QLabel("Калькулятор Жизни", this);
    title->setAlignment(Qt::AlignCenter);
    QFont font = title->font();
    font.setPointSize(16);
    font.setBold(true);
    title->setFont(font);
    mainLayout->addWidget(title);

    m_inputButton = new QPushButton("Ввести данные", this);
    m_inputButton->setMinimumHeight(40);
    mainLayout->addWidget(m_inputButton);

    connect(m_inputButton, &QPushButton::clicked,
           this, &MainWindow::openInputRequested);

    QGroupBox *resultsGroup = new QGroupBox("Результаты", this);
    QVBoxLayout *resultsLayout = new QVBoxLayout(resultsGroup);

    m_sleepLabel = new QLabel("Сон: -", this);
    m_blinksLabel = new QLabel("Моргания: -", this);
    m_heartLabel = new QLabel("Удары сердца: -", this);
    m_bloodLabel = new QLabel("Перекачено крови: -", this);
    m_waterLabel = new QLabel("Выпито воды: -", this);
    mLaughsLabel = new QLabel("Смех: -", this);

    resultsLayout->addWidget(m_sleepLabel);
    resultsLayout->addWidget(m_blinksLabel);
    resultsLayout->addWidget(m_heartLabel);
```

```

        resultsLayout->addWidget(m_bloodLabel);
        resultsLayout->addWidget(m_waterLabel);
        resultsLayout->addWidget(m_laughsLabel);

        mainLayout->addWidget(resultsGroup);

        m_statusLabel = new QLabel("Введите дату рождения для расчёта",
this);
        m_statusLabel->setAlignment(Qt::AlignCenter);
        mainLayout->addWidget(m_statusLabel);

        setCentralWidget(central);
    }

void MainWindow::onModelChanged() {
    updateResults();
}

void MainWindow::updateResults() {
    m_sleepLabel->setText(QString(" Сон: %1 часов (%2 суток)")
        .arg(m_model->sleepHours(), 0, 'f', 0)
        .arg(m_model->sleepDays(), 0, 'f', 1));

    m_blinksLabel->setText(QString(" Моргания: %1 раз")
        .arg(m_model->blinks()));

    m_heartLabel->setText(QString(" Удары сердца: %1 раз")
        .arg(m_model->heartBeats()));

    m_bloodLabel->setText(QString(" Перекачано крови: %1 литров")
        .arg(m_model->bloodLiters(), 0, 'f', 0));

    m_waterLabel->setText(QString(" Выпито воды: %1 литров")
        .arg(m_model->waterLiters(), 0, 'f', 0));

    m_laughsLabel->setText(QString(" Смех: %1 раз")
        .arg(m_model->laughs()));

    m_statusLabel->setText(" Расчёт выполнен успешно!");
}

void MainWindow::showSuccessMessage(const QString &msg) {
    m_statusLabel->setText(msg);
}

void MainWindow::setModel(LifeModel *model) {
    m_model = model;
    m_model->addObserver(this);
}

```

mainwindow.h

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include "lifemodel.h"

class QLabel;
class QPushButton;

class MainWindow : public QMainWindow, public ModelObserver {
    Q_OBJECT

public:
    explicit MainWindow(QWidget *parent = nullptr);

    void onModelChanged() override;

    void setModel(LifeModel* model);

    void showSuccessMessage(const QString &msg);

signals:
    void openInputRequested();

private:
    void createUI();
    void updateResults();

    QLabel *m_sleepLabel;
    QLabel *m_blinksLabel;
    QLabel *m_heartLabel;
    QLabel *m_bloodLabel;
    QLabel *m_waterLabel;
    QLabel *m_laughLabel;
    QLabel *m_statusLabel;

    QPushButton *m_inputButton;

    LifeModel *m_model;
};

#endif
```